Why Care About Technical Debt?

Quick fix solutions in software design, if not corrected, can lead to 'technical debt', which if not addressed in time, can lead to 'technical bankruptcy'. This article, which is an extract from the first chapter of the book, 'Refactoring for Software Design Smells: Managing Technical Debt', looks into the causes and impact of technical debt, and gives a few tips on how to manage it.

Technical debt is the debt that accrues when you knowingly or unknowingly make wrong or non-optimal design decisions.

Technical debt is a term coined by Ward Cunningham in a 1992 report. It is analogous to financial debt. When a person takes a loan (or uses his credit card), he incurs debt. If he regularly pays the installments (or the credit card bill) then the created debt is repaid and does not lead to further problems. However, if the person does not pay his installments (or bills), a penalty in the form of interest is applicable and this mounts every time he misses making a payment. In case the person is not able to pay the installments (or bills) for a long time, the accrued interest can make the total debt so ominously large that the person may have to declare bankruptcy.

Along the same lines, when software developers opt for a quick fix rather than a proper well-designed solution, they introduce technical debt. It is okay if the developers pay back the debt on time. However, if they choose not to or forget about the debt created, the accrued interest on the technical debt piles up, just like financial debt. The debt keeps increasing over time with each change to the software; thus, the later the developers pay off the debt, the more expensive it is to pay off. If the debt is not paid at all, then eventually, the pile-up is so huge that it becomes immensely difficult to change the software. In extreme cases, the accumulated technical debt is too big to ever be paid off and the product has to be abandoned. Such a situation is called technical bankruptcy.

What constitutes technical debt?

There are multiple sources of technical debt (see Figure 1). Some of its well-known dimensions include (with examples):

- *Code debt:* Static analysis tool violations and inconsistent coding style.
- *Design debt:* Design smells and violations of design rules.
- *Test debt:* Lack of tests, inadequate test coverage, and improper test design.
- *Documentation debt:* No documentation for important concerns, poor documentation and outdated documentation.

This book is primarily concerned with the design aspects of technical debt, i.e., design debt. In other words, when the author refers to technical debt in this book, he implies design debt.

To better understand design debt, let us take the case of a medium-sized organisation that develops software products. To be able to compete with other organisations in the market, the former obviously needs to release newer products into the market faster and at reduced costs. But how does this impact its software development process?